NoteMind AI

Frontend API Reference

> **Base URL:** `http://localhost:8080/api/v1`
> **Authentication:** Every endpoint except `/auth/**` requires `Authorization: Bearer <jwtToken>`.
> **Source of truth:** Spring controllers and DTOs in the current backend.
> **Generated:** 2026-10-14

Universal response wrapper

```
{
  "success": true,
  "message": "Success",
  "data": { },
  "timestamp": "2026-10-14T18:28:14.849Z"
}
```

The `data` property is omitted when null. Error responses use `success: false`, an error message, and no data.

1. Authentication

Authentication endpoints are public. Store both tokens returned by login/signup. Send the JWT access token in the Bearer header for protected requests.

POST /auth/signup

Create an account. **Success:** 200

```
Request JSON
{ "username": "john_doe", "password": "securePass123" }

Response data
{ "username": "john_doe", "userId": 1, "jwtToken": "<jwt>", "refreshToken": "<refresh-token>" }
```

POST /auth/login

Log in. **Success:** 200

```
Request JSON
{ "username": "john_doe", "password": "securePass123" }

Response data
{ "username": "john_doe", "userId": 1, "jwtToken": "<jwt>", "refreshToken": "<refresh-token>" }
```

POST /auth/refresh-token

Exchange a refresh token for new tokens. **Success:** 200

```
Request JSON
{ "refreshToken": "<refresh-token>" }

Response data
{ "refreshToken": "<new-refresh-token>", "jwtToken": "<new-jwt>" }
```

POST /auth/logout

Revoke a refresh token. **Success:** 200

```
Request JSON
{ "refreshToken": "<refresh-token>" }

Response
{ "success": true, "message": "Logged out successfully", "timestamp": "..." }
```

2. Notes

Note fields: `id`, `title`, `content`, `summary`, `summaryType`, `imageUrl`, `createdAt`, `updatedAt`, `tags`, and `folderId`. Dates are ISO date-time strings.

POST `/notes`

Create a note. **Content-Type:** `multipart/form-data`. **Success:** 200

Part	Required	Format
`note`	Yes	JSON text, preferably with part content type `application/json`
`image`	No	Image file

```
note part JSON
{
  "title": "Quantum Computing",
  "content": "Quantum computers use qubits...",
  "summaryType": "DETAILED",
  "folderId": 1,
  "tags": [{ "name": "technology" }]
}

Response data
{
  "id": 1, "title": "Quantum Computing", "content": "Quantum computers use qubits...",
  "summary": "AI-generated summary", "summaryType": "DETAILED", "imageUrl": null,
  "createdAt": "2026-10-14T18:28:14", "updatedAt": "2026-10-14T18:28:14",
  "tags": [{ "id": 1, "name": "technology" }], "folderId": 1
}
```

GET `/notes`

List notes. **Success:** 200

Query parameters: `page` (zero-based, default 0), `size` (default 10), `sort` (for example `createdAt,desc`; controller default sort is `createdAt`).

```
Response data
{
  "content": [{ "id": 1, "title": "...", "content": "...", "summary": "...",
    "summaryType": "SHORT", "imageUrl": null, "createdAt": "...", "updatedAt": "...",
    "tags": [], "folderId": null }],
  "pageable": { "pageNumber": 0, "pageSize": 10, "offset": 0, "paged": true, "unpaged": false },
  "totalPages": 1, "totalElements": 1, "last": true, "size": 10, "number": 0,
  "numberOfElements": 1, "first": true, "empty": false
}
```

GET /notes/{id}

Get one note. **Success:** 200. Example: `/notes/1`.

```
Response data
{ "id": 1, "title": "...", "content": "...", "summary": "...", "summaryType": "SHORT",
  "imageUrl": null, "createdAt": "...", "updatedAt": "...", "tags": [], "folderId": null }
```

PUT /notes/{id}

Update a note. Same multipart format as create. **Success:** 200.

DELETE /notes/{id}

Delete a note. **Success:** 200.

```
Response
{ "success": true, "message": "Note deleted successfully", "timestamp": "..." }
```

GET /notes/search?query={text}

Search notes. **Success:** 200. Response data is `NoteDto[]`.

GET /notes/refresh-summary/{id}?type={summaryType}

Regenerate a summary. **Success:** 200. Allowed types: `SHORT`, `DETAILED`, `BULLET_POINTS`.

3. Folders

POST `/folder`

Create a folder. **Success:** 200.

```
Request JSON: { "name": "Study" }
Response data: { "id": 1, "name": "Study" }
```

GET `/folder?includeNotes={boolean}`

List folders. `includeNotes` defaults to `false`. **Success:** 200.

```
includeNotes=false response data:
[{ "id": 1, "name": "Study" }]

includeNotes=true response data:
[{ "id": 1, "name": "Study", "notes": [ /* NoteDto[] */ ] }]
```

GET `/folder/{id}`

Get a folder without notes. **Success:** 200. Response data is `{ "id": 1, "name": "Study" }`.

PATCH `/folder/{id}`

Rename a folder. **Success:** 200. Request: `{ "name": "Exam Prep" }`.

DELETE `/folder/{id}`

Delete a folder. **Success:** 200. Message: `Folder deleted successfully`.

4. Flashcards

POST `/notes/{noteId}/flashcards/generate?count={n}`

Generate AI flashcards. `count` defaults to 5 and must be 1-50. **Success:** 201. Response data is `FlashCardDto[]`.

POST `/notes/{noteId}/flashcards/regenerate?count={n}`

Replace/regenerate AI flashcards. Count defaults to 5 and must be 1-50. **Success:** 200. Response data is `FlashCardDto[]`.

GET `/notes/{noteId}/flashcards`

List flashcards for a note. **Success:** 200. Response data is `FlashCardDto[]`.

GET `/flashcards/{flashCardId}`

Get one flashcard. **Success:** 200.

POST `/notes/{noteId}/flashcards`

Create a flashcard. **Success:** 201. Request JSON: `{ "question": "What is a qubit?", "answer": "A quantum bit." }`.

PATCH `/flashcards/{flashCardId}`

Update a flashcard. **Success:** 200. Request JSON uses `question` and `answer`.

DELETE `/flashcards/{flashCardId}`

Delete one flashcard. **Success:** 200.

DELETE `/notes/{noteId}/flashcards`

Delete all flashcards for a note. **Success:** 200.

```
FlashCardDto
{ "id": 1, "question": "What is a qubit?", "answer": "A quantum bit.",
  "noteId": 1, "createdAt": "2026-10-14T18:28:14" }
```

5. Quizzes

POST `/notes/{noteId}/quiz/generate?count={n}`

Generate an AI quiz. `count` defaults to 5 and must be 1-20. **Success:** 201.

```
Response data
[{
  "question": "Which statement is correct?",
  "options": ["Option A", "Option B", "Option C", "Option D"],
  "correctAnswer": "Option B",
  "explanation": "Why Option B is correct."
}]
```

6. Errors and frontend notes

Status	Meaning	Typical cases
400	Bad request	Invalid note/folder, empty search, validation or malformed input
401	Unauthorized	Invalid credentials, expired/invalid refresh token, missing login
404	Not found	User, note, folder, or flashcard does not exist
409	Conflict	Username or folder name already exists
500	Server error	Unexpected error or image upload failure
502	AI gateway error	AI service failure while generating content

```
Error response shape
{
  "success": false,
  "message": "Human-readable error message",
  "timestamp": "2026-10-14T18:28:14.849Z"
}
```

- Use the access token as `Authorization: Bearer <jwtToken>`; do not send the refresh token in this header.
- On a 401 from a protected request, call `/auth/refresh-token` once, replace stored tokens, then retry the original request.
- Multipart note requests must send the JSON note as the `note` part and the optional file as `image`.
- Generated flashcards and quizzes return arrays inside `data`; do not expect a bare array.

This document describes the current backend implementation. Confirm deployment host and CORS settings before connecting a production frontend.